

Olimpo: o que um classificador linear aprende sobre notícias falsas em português, e como isso alimenta uma plataforma educativa

Equipe Olimpo
PUC-Campinas & Instituto de Pesquisas ELDORADO

Outubro de 2026

Resumo

O Olimpo é uma plataforma gamificada que treina o reconhecimento de notícias falsas por meio de votação em salas e feedback socrático. Este artigo resume as conclusões da trilha de aprendizado de máquina que sustenta o produto e o que, ao final, está de fato em uso. No Fake.br-Corpus (7.200 notícias), um *LinearSVC* sobre TF-IDF de palavras e caracteres atinge $F1 \approx 0,93$ em validação cruzada, e nem redução de dimensionalidade, nem kernels, nem árvores, nem metadados morfossintáticos trazem ganho relevante. Em títulos do Fake-Recogna, porém, o $F1$ cai para $\approx 0,61$ e o AUC para $0,66\text{--}0,71$: parte do sinal é artefato da fonte (autoria, tipografia, tamanho). Os métodos não supervisionados mostraram-se úteis para *descrever* estilos de escrita, não para classificar. O modelo em produção é um *LinearSVC* calibrado com atributos spaCy, χ^2 e SVD, cuja pontuação se decompõe exatamente em n-gramas e atributos. Ele já roda em um serviço Python próprio, consumido pelo backend do jogo, e seu resultado é revelado só após o fechamento coletivo de cada rodada e tratado como *indício*, não como veredito. Um segundo componente, baseado em regras do FP-Growth, mostra durante a leitura como a escrita do trecho se compara à de cada classe do corpus, sem classificar.

Palavras-chave: desinformação, classificação de texto, SVM, FP-Growth, interpretabilidade, educação midiática.

1 Introdução

Plataformas digitais priorizam engajamento sobre precisão, e ferramentas de checagem que apenas carimbam um texto como “falso” tendem a ser rejeitadas por quem já discorda delas. A proposta do Olimpo é outra: em salas multiplayer ou no modo solo, os jogadores votam entre *Fake*, *Fato real* e *Tenho dúvida*, e ao fim da rodada recebem o gabarito com uma justificativa pedagógica e perguntas no estilo socrático. O componente de IA fornece a classificação e os motivos usados nesse feedback.

A pergunta que guia o trabalho técnico é: *o que um modelo treinado em um corpus rotulado de notícias consegue, de fato, afirmar sobre textos novos?* A resposta condiciona como o produto deve apresentar a saída do modelo.

2 Dados e protocolo

Corpus. Usamos o Fake.br-Corpus (NILC/USP): 7.200 notícias em português, 3.600 falsas e 3.600 verdadeiras. As falsas são bem mais curtas (mediana de 157 palavras contra 918). Para

checar generalização, usamos o FakeRecogna (11.903 registros), restrito a *títulos*: 2.484 falsos do domínio `boatos.org` e igual número de reais, com mediana de 13 palavras.

Pré-processamento. Normalização tipográfica (aspas, travessões, reticências, dígitos $\rightarrow$ 0), truncamento em 100 palavras e metadados em taxa por 100 palavras. Essas decisões vieram da análise exploratória: sem elas o modelo aprendia a fonte em vez do conteúdo. Partição estratificada 80/20 (5.760 treino, 1.440 teste) e validação cruzada de 5 folds com a métrica F1.

Modelos. Supervisionados: *LinearSVC*, Regressão Logística (baseline interpretável) e Random Forest, sobre TF-IDF de palavras e caracteres, com e sem atributos morfossintáticos extraídos por spaCy. Não supervisionados: Isolation Forest, LOF, One-Class SVM, K-means, DBSCAN, HDBSCAN e mineração de regras de associação com FP-Growth.

3 Resultados

3.1 Artefatos de fonte no corpus

Três achados independentes mostram que parte do sinal vem da fonte, e não da linguagem da desinformação:

- **Autoria:** 3.528 de 3.601 notícias sem autor são falsas e 3.527 de 3.599 com autor são reais (Cramér $V = 0,960$). O campo foi excluído dos modelos.
- **Formatação:** no texto bruto, espaços e quebras são 2,93% dos tokens das falsas contra 0,38% das verdadeiras. Após normalização e truncamento, caem a 0,00% nas duas classes.
- **Convenções de redação:** marcadores temporais (“nesta”, “quarta”) concentram-se nas verdadeiras e afetam a classificação.

Resta um sinal de tamanho: 17% das falsas têm menos de 100 palavras, contra $\approx 0\%$ das verdadeiras.

3.2 Aprendizado supervisionado

Configuração	F1 (CV, 5 folds)	Observação
<i>LinearSVC</i> word+char, $C=1$	$0,9295 \pm 0,0099$	referência interna
Regressão Logística word+char	$0,9182$ ($C=1$) $\rightarrow$ $0,9281$ ($C=10$)	baseline interpretável
Random Forest word+char	$0,9285$	protocolo anterior
Redução (χ^2 , L1, SVD)	—	nenhuma supera a base
Kernels RBF/polinomial	—	sem ganho
+ metadados spaCy ($w=0,03$)	$0,9344$	+0,005; ganho frágil
spaCy + χ^2 10 mil + SVD(500)	$0,9295$	gap 0,042 (base: 0,0705)

Tabela 1: Desempenho interno no Fake.br-Corpus.

Em síntese: (i) em regime de muitos atributos e poucos documentos, o modelo linear simples é difícil de superar, e $C=1$ é o ponto ótimo; (ii) reduzir a dimensão só diminui a distância entre treino e validação (por ajuste menor), sem melhorar F1; (iii) os metadados do corpus tiveram efeito contraditório (−2,4 pt na Regressão Logística, +1,4 pt no SVM), e os atributos spaCy somam apenas +0,005 e só com peso baixo; (iv) no teste interno, spaCy e base empatam dentro do erro padrão ($\approx \pm 0,007$).

3.3 Generalização fora do corpus

Conjunto	Modelo	F1 macro	ROC-AUC	% previsto fake
Interno (Fake.br)	base word+char	0,9222	0,9830	49,7
Interno (Fake.br)	+ spaCy	0,9236	0,9846	49,6
Externo (FakeRecogna, títulos)	base word+char	0,6119	0,7072	70,8
Externo (FakeRecogna, títulos)	+ spaCy	0,6134	0,6622	59,3

Tabela 2: Teste interno e teste externo (treino sempre no Fake.br).

O desempenho não se transfere. O modelo tende a rotular como falso o que é curto e diferente do corpus, e as variáveis de maior peso (taxa e tamanho médio de sentenças) saem da distribuição de treino em títulos. Os atributos spaCy deslocam o limiar, mas *pioram* a ordenação (AUC 0,707 $\rightarrow$ 0,662). Nenhuma técnica de dimensionalidade resolveu a distância de domínio e formato.

3.4 Aprendizado não supervisionado

Os detectores de novidade atingiram acurácias altas (Isolation Forest 0,97; DBSCAN 0,97; LOF 0,95), mas a leitura foi revisada: eles aprendem desvio em relação às notícias verdadeiras, e como a autoria separa quase por completo as classes, o resultado reflete o corpus, não a falsidade. Em holdout temporal a acurácia cai (Isolation Forest 0,86; DBSCAN 0,82). O agrupamento temático teve silhouette de 0,008 (K-means) e 0,040 (HDBSCAN), sem autorizar tratar grupos como classes de veracidade.

O modelo principal atual é o FP-Growth sem autoria da raiz de `machine-learning/unsupervised-learning/`: `fp_growth_principal.py` e `fp_growth_principal_sem_autoria.ipynb`, variante `sintaxe ampliada`. No run `fp-growth-metadados-ampliados-20261008T002311Z`, são 22 atributos ativos, 44 itens, 523 regras direcionais elegíveis e 345 padrões distintos. Na validação, 422 regras mantêm os filtros e 347 também têm redescoberta $\geq 80\%$ em 100 reamostragens dos grupos de treino. O teste conhecido é exploratório. O método descreve coocorrências e não classifica a veracidade de notícias.

Os quantis e regras usam apenas o treino canônico (4.320 notícias); validação e teste têm 1.440 notícias cada e preservam grupos de pares e duplicatas. A extração usa os primeiros 300 caracteres após normalização NFKC. O suporte mínimo é 0,08 e os filtros são confidence $\geq 0,50$, lift $\geq 1,05$ e Jaccard $\geq 0,10$. Classe e autoria são usadas apenas na descrição posterior; o ranking de 20 candidatos não usa rótulos. Os relatórios estilísticos e o ranking anterior com 25 padrões são históricos e não representam este principal.

Na validação, 105 dos 345 padrões têm maior frequência dentro da classe Fake, 235 dentro de True e cinco empatam. Essas frequências não são probabilidades de falsidade. A união de todos os padrões cobre 1.439/1.440 notícias; a fila de revisão cobre 1.346, sem comprovar eficácia pedagógica. Ainda faltam avaliação externa e revisão editorial humana.

4 O que está em uso

A trilha de pesquisa chegou ao produto por uma camada de execução separada, o `model-engine`, que é consumida pelo servidor Next.js. Os dois componentes de IA têm funções, saídas e momentos de exibição distintos e propositalmente *não* são combinados em um único escore.

Componente	Função	Onde aparece no jogo
Observação de escrita (FP-Growth)	Mostra padrões gramaticais presentes no trecho e a frequência da mesma combinação em cada classe do corpus	Painel “Observe a escrita”, durante a leitura
Modelo supervisionado	Estima $P(\text{fake})$ com explicação por n-grama e atributo	“Análise do modelo”, após todos votarem ou o tempo acabar
Gabarito cadastrado	Referência da pontuação dos jogadores	Veredito da rodada, identificado à parte da previsão

Tabela 3: Papéis dos componentes no jogo.

4.1 Modelo supervisionado

Modelo. *LinearSVC* ($C=1$, classes balanceadas) com 24 atributos spaCy (peso 0,03), seleção χ^2 de 10 mil n-gramas e SVD de 500 dimensões, calibrado por sigmoide (versão `svm-spacy-chi2k10k-svd500-v1`). Iguala a base em F1 de validação cruzada (0,9295), reduz o gap de 0,0705 para 0,042 e tem F1 de 0,9243 no teste interno sem calibração. A versão calibrada registra F1 de 0,9208 e AUC de 0,9789 antes do reajuste final. A escolha se deve menos ao desempenho que à *explicabilidade*: o escore se decompõe exatamente em contribuições por n-grama e por atributo de estilo, que viram os motivos exibidos. O artefato final foi reajustado em treino mais teste (7.200 notícias), de modo que as métricas internas citadas vêm de outro ajuste e não avaliam esse artefato de forma independente.

Política de decisão. O escore é $100 \times P(\text{fake})$. A classificação é *confiável* para $P \leq 0,35$, *não confiável* para $P \geq 0,65$ e *incerta* entre os dois valores. Textos com menos de 30 palavras não são classificados, e a análise usa as primeiras 100 palavras do corpo extraído, como no treino. Os limiares são decisão de produto.

Serviço. O `model-engine` é um serviço HTTP em Python (`POST /supervised/analyze`), que recebe apenas o corpo da notícia e não registra seu conteúdo. Antes de carregar o modelo, confere o hash SHA-256 do artefato, o patch exato do Python (3.14.2) e as versões de scikit-learn, spaCy e demais bibliotecas; se algo divergir, somente o supervisionado fica indisponível, e as observações de escrita continuam funcionando. Numa medição local (20 análises aquecidas), a mediana foi de 140,6 ms e o p95 de 188,3 ms, com pico de 438 MiB de memória; com oito requisições simultâneas, três foram atendidas e cinco recusadas, pois a fila é intencionalmente curta (uma execução e duas vagas de espera). Essa medição não representa um teste de carga em produção.

Integração no backend. Um adaptador HTTP substituiu o *mock* como implementação padrão do contrato de análise (`classification`, `escore`, `reasons`, `modelVersion`). A resposta é guardada em PostgreSQL com chave formada pelo conteúdo, pelo hash do artefato, pelo código de inferência e pela política, de modo que trocar a versão do modelo gera nova análise. Em um teste com quatro consultas concorrentes de duas instâncias do caso de uso houve uma única inferência e um único registro.

Acesso e pontuação. A rota `POST/api/news-prediction` exige sessão, participação na sala e rodada encerrada coletivamente: votar, avançar ou concluir individualmente não libera a previsão. Votos, placar e eventos em tempo real fecham sem chamar o modelo, e a pontuação continua vindo do gabarito cadastrado. Se o modelo falhar, a partida segue normalmente e apenas a previsão fica indisponível.

Validação. Um teste ponta a ponta com parser HTML real, modelo real, banco e Redis descartáveis e duas sessões de navegador verificou o bloqueio antes do fechamento, as cinco consultas

concorrentes com um registro, a recarga, as telas desktop e celular, e o encerramento da partida com falha induzida do modelo. A suíte do web app reportou 419 testes aprovados, e a suíte Python 20, incluindo reprodução numérica das *features* e das probabilidades do código original.

4.2 Observação de escrita (FP-Growth)

O FP-Growth foi levado ao produto como *descrição*, não como classificador. O catálogo em uso vem da variante `sintaxe_ampliada` (22 atributos, 523 regras direcionais elegíveis e 345 padrões distintos) e contém 20 padrões candidatos, selecionados sem usar classe ou autoria no ranking. Para um texto novo, o motor normaliza e recorta os primeiros 300 caracteres, extrai as medidas com limites congelados, sem recalcular quantis nem receber gabarito ou autor, e indica quais padrões correspondem *integralmente*. Cada padrão traz uma explicação em linguagem simples (por exemplo, “afirmações ligadas a um verbo, como *disse que o prazo acabou*”) e a frequência da mesma combinação em cada classe da partição de validação (720 notícias falsas e 720 verdadeiras). Essas frequências são *dentro de cada classe* do corpus, e não a probabilidade de a notícia ser falsa. O catálogo está marcado como `experimental_descriptive_comparison` e não passou por aprovação editorial humana.

4.3 Plataforma

Aplicação Next.js 16/React 19 em TypeScript, com PostgreSQL (Drizzle), Redis e eventos por SSE, organizada em Clean Architecture no backend e Feature-Sliced Design no frontend. Estão implementados autenticação, salas com PIN, votação em três níveis, ranking, desafios autorais, extração de notícias por URL com proteção contra SSRF e a interface por etapas. O ambiente completo (app, migrações, banco, Redis e `model-engine`) sobe com um único `docker compose`. A reflexão socrática exibida na interface ainda consiste em perguntas gerais fixas; a geração dinâmica de perguntas a partir dos motivos do modelo e dos padrões observados não existe.

5 Discussão e limitações

1. **Dentro do corpus, o problema está resolvido por um modelo simples; fora dele, não.** $F1 \approx 0,93$ contra $\approx 0,61$ em títulos de outra fonte.
2. **Rótulo e fonte estão confundidos.** Há um único corpus de treino, rotulado por origem, e o desempenho alto parcialmente mede reconhecimento da fonte.
3. **Complexidade não comprou generalização.** Kernels, árvores, redução de dimensão e metadados não ajudaram; o ganho do modelo de produção é de explicabilidade e de menor ajuste ao treino.
4. **Cuidados de avaliação.** As partições supervisionadas são aleatórias (pares temáticos podem cruzar treino e teste), e cada configuração externa foi avaliada uma única vez. As avaliações do FP-Growth descrevem o corpus, não medem generalização.
5. **Sem avaliação em uso real.** O modelo funciona de ponta a ponta, mas nenhuma métrica foi medida em notícias submetidas por usuários, e a transferência externa é fraca ($AUC \approx 0,68$ em títulos). A interface não deve exibir números sem origem medida, como a margem de erro e a dica de “78%” que estavam fixas na tela.
6. **Limites operacionais.** O marcador de fechamento da rodada no Redis expira em 24 horas; depois disso a previsão é bloqueada, mesmo em partidas finalizadas. A fila de inferência é curta e recusa picos de requisições simultâneas.

Esses limites sustentam o posicionamento do produto: o Olimpo é uma plataforma *educacional* que apresenta indícios, e não um detector definitivo de fake news; o modelo não verifica fatos.

6 Conclusão

O trabalho estabeleceu que um classificador linear sobre TF-IDF resolve o Fake.br-Corpus com $F1 \approx 0,93$, que esse resultado é em boa parte específico do corpus e que técnicas mais sofisticadas não o melhoram nem o transferem. O valor mais robusto da trilha foi metodológico: identificar e neutralizar artefatos de fonte, separar descoberta de classificação e escolher um modelo cuja saída pode ser explicada ao jogador. O resultado prático é que a pesquisa chegou ao jogo: o modelo supervisionado roda em serviço próprio, com versão e hash verificados, e é revelado só depois do fechamento coletivo; o FP-Growth acompanha a leitura como descrição de escrita. Os próximos passos são: (1) medir o modelo com notícias reais extraídas pela plataforma; (2) coletar um conjunto externo de texto completo com fontes e épocas distintas e repetir a avaliação; (3) submeter o catálogo de observações a revisão editorial humana e a um estudo de compreensão com jogadores; (4) gerar dinamicamente as perguntas socráticas a partir dos motivos do modelo e dos padrões observados; e (5) persistir o fechamento da rodada de forma durável e dimensionar a fila de inferência para carga real.

Referências

- NILC/USP. *Fake.br-Corpus*. Revisão fixa 780f5516c4ae070761632d98ac3368f3ded09d35.
- RECOGNA NLP. *FakeRecogna*. <https://huggingface.co/datasets/recogna-nlp/FakeRecogna>.
- Lundberg, S.; Lee, S.-I. A unified approach to interpreting model predictions (SHAP). <https://shap.readthedocs.io/>.
- Scikit-learn. *User guide*. <https://scikit-learn.org/stable/>.
- Equipe Olimpo. *Documento técnico consolidado* (EntregávelFinal-Completo) e relatórios em `machine-learning/`, `model-engine/` e `docs/machine-learning/validacao-integracao-supervisionado.md` (branch dev), repositório olimpo-fake-news-ai, 2026.